REST ASSURED
CLASS 8

Static & Dynamic Request Payloads
Serialization • Deserialization • Faker • Nested POJOs

Beginner-friendly study notes with line-by-line code explanations, realistic API testing scenarios, hands-on tasks, worked solutions, and best practices.

Based on your uploaded "Rest Assured class 8.txt"

Learning target: Start from a stored JSON fixture, map it to a POJO, selectively replace test data with Faker values, serialize it back to JSON, send it with Rest Assured, and validate the API response.

How to use these notes

Follow the chapters in order. The class file moves from a simple Comments payload to employee creation and then to a nested User payload. This workbook keeps that learning progression and adds explanations and practice around the same patterns.

Chapter	What you will learn
1. Static vs dynamic payload	When to send a JSON file as-is and when to modify its values.
2. Serialization concepts	JSON → Java object (deserialization) and Java object → JSON (serialization).
3. Comments API	Read a fixture, map to Comments POJO, generate values, POST request.
4. Faker	Generate names, emails, addresses and test-safe data.
5. Employee API	Update selected fields in an employee fixture before sending it.
6. Nested User payload	Model address, name and geolocation objects, including JSON field mapping.
7. Assertions & negative tests	Validate response code and values, diagnose failures.
8. Best practices & exercises	Build repeatable tests and practise independently.

Prerequisites

- Basic Java: classes, objects, variables, methods and exceptions.
- Basic REST concepts: POST request, URL, headers, JSON body and HTTP status codes.
- A Maven project with Rest Assured, Jackson, TestNG (optional but recommended), Lombok (optional), and JavaFaker.
- A running local employee API for the localhost example, or a public practice API for demonstration.

Important source detail: The uploaded class uses JSONPlaceholder, FakeStore API, and a local endpoint at http://localhost:9192/api/employees. These are examples from the notes; the workbook does not claim to have executed the requests. Public demo APIs may simulate creates rather than persist data permanently.

1. Static request payload vs dynamic request payload

1.1 Static payload

A static payload is test data that you send without changing its field values. You keep the JSON in a file and pass that File to Rest Assured.

```
File payloadFile = new File("src/test/resources/payload/commentsPostPayload.json");

Response response = given()
   .contentType(ContentType.JSON)
   .body(payloadFile)
.when()
   .post("https://jsonplaceholder.typicode.com/comments");
```

Use this approach when a known, fixed example is enough: checking a required-field rule, reproducing a defect, or keeping a stable regression test fixture. Static data is easy to understand and debug, but repeated tests can collide if the API requires unique emails or usernames.

1.2 Dynamic payload

A dynamic payload changes some or all fields for each test execution. In your class, the recommended flow is: JSON fixture → POJO → Faker-generated values → POJO updated → JSON string → HTTP request.

```
File file = new File("src/test/resources/payload/employeePostRequestBody.json");
ObjectMapper mapper = new ObjectMapper();

Employees employee = mapper.readValue(file, Employees.class); // JSON -> POJO
Faker faker = new Faker();
employee.setFirstName(faker.name().firstName());
employee.setLastName(faker.name().lastName());
employee.setEmail(faker.internet().emailAddress());

String requestJson = mapper.writeValueAsString(employee); // POJO -> JSON
```

1.3 Decision table

Situation	Recommended approach	Why
Need exactly the same request each run	Static JSON file	Reproducible, easy to inspect.
API rejects duplicate email addresses	Fixture + dynamic email	Preserve known-good structure but avoid collisions.
Need many boundary values	POJO builder / data provider	Control each test case deliberately.
Need to test malformed JSON syntax	Raw string or file	POJO serialization will usually produce valid JSON.
Need nested payloads with arrays	Nested POJO + List fields	Clear mapping between Java model and JSON structure.

Best practice: Do not make every field random. Generate only the fields that should vary; keep invariant business rules explicit. Random data that violates the API contract creates noise rather than useful coverage.

2. Serialization and deserialization

2.1 The simple mental model

JSON is text transferred over HTTP. Java code usually works with objects. Jackson's ObjectMapper converts between the two forms.

Operation	Direction	Jackson method	Example
Deserialization	JSON file/text → Java object	readValue(...)	Load Comments from commentsPostPayload.json.
Serialization	Java object → JSON text	writeValueAsString(...)	Convert modified Comments back to JSON before POST.

2.2 Deserialization

```
ObjectMapper mapper = new ObjectMapper();
File jsonFile = new File("src/test/resources/payload/commentsPostPayload.json");
Comments comments = mapper.readValue(jsonFile, Comments.class);
```

- ObjectMapper mapper = new ObjectMapper(); creates the Jackson converter.
- new File(...) points to the fixture on disk. The path is relative to the project working directory in this example.
- readValue(...) parses JSON, matches field names to Java properties, and creates a Comments object.
- Comments.class tells Jackson which Java type it should construct.

2.3 Serialization

```
comments.setName("Mira Shah");
comments.setEmail("mira.qa@example.test");
String json = mapper.writeValueAsString(comments);
```

The setters update the in-memory object. writeValueAsString() asks Jackson to read the object properties and create JSON text representing the object's current state. That string can be provided to Rest Assured body().

2.4 The full round trip

```
commentsPostPayload.json
      | readValue(file, Comments.class)
      v
  Comments POJO  <--- setters / Faker can modify fields
      | writeValueAsString(comments)
      v
  JSON string
      | request.body(jsonString)
      v
  HTTP POST request
```

Common confusion: Serialization is not the HTTP request itself. It is the conversion of an object to JSON. Rest Assured then sends that JSON as the request body.

2.5 What is a POJO?

POJO means Plain Old Java Object. It is a regular Java class used here to represent a JSON structure. Jackson maps JSON fields onto Java fields/properties using JavaBean getters/setters or field visibility and configured mapping rules.

3. Project structure and dependencies

3.1 Suggested folder layout

```
your-project/
 pom.xml
 src/test/java/
  pojo/Comments.java
  pojo/Employees.java
  tests/CommentsApiTest.java
  tests/EmployeeApiTest.java
 src/test/resources/
  payload/commentsPostPayload.json
  payload/employeePostRequestBody.json
```

Use src/test/resources for test fixtures instead of keeping JSON files inside src/test/java. This keeps data separate from executable code. For a larger framework, place shared test utilities in a test utility package.

3.2 Maven dependency fragments

The class file includes JavaFaker version 1.0.2. Add compatible versions of the other dependencies in your project or use versions managed by your framework parent POM. The following are dependency fragments; define version properties in your POM or substitute approved versions for your project.

```
<dependencies>
 <dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>${rest-assured.version}</version>
  <scope>test</scope>
 </dependency>

 <dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>${jackson.version}</version>
  <scope>test</scope>
 </dependency>

 <dependency>
  <groupId>com.github.javafaker</groupId>
  <artifactId>javafaker</artifactId>
  <version>1.0.2</version>
  <scope>test</scope>
 </dependency>

 <dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>${testng.version}</version>
  <scope>test</scope>
 </dependency>

 <dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>${lombok.version}</version>
  <scope>provided</scope>
 </dependency>
</dependencies>
```

Lombok is optional: If you use @Data, make sure Lombok is correctly configured in the IDE/build. If you do not want Lombok, write getters/setters manually. Do not add Lombok merely to avoid understanding what the generated methods do.

3.3 Use relative paths

```
Path payloadPath = Paths.get(
   "src", "test", "resources", "payload", "commentsPostPayload.json");
```

```
File payloadFile = payloadPath.toFile();
```

Avoid personal absolute paths such as C:\Users\someone\... in shared tests. They work only on one computer. A relative project path or classpath resource makes the test portable across teammates and CI agents.

4. Example 1 — Comments API, fixture to dynamic POST

4.1 Original fixture structure

```
{
 "postId": 101,
 "name": "Amol",
 "email": "amol@gmail.com",
 "body": "this is basic content"
}
```

4.2 Comments POJO

```
package pojo;

import lombok.Data;

@Data
public class Comments {
   private Integer postId;
   private String name;
   private String email;
   private String body;
}
```

@Data generates getters, setters, toString, equals and hashCode methods. Integer is shown instead of int because it can represent null when a field is absent; a primitive int defaults to 0 and may hide a missing field. Choose the type based on your API contract.

4.3 Complete Rest Assured example

```
package tests;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.equalTo;
import static org.hamcrest.Matchers.notNullValue;

import java.io.File;
import java.io.IOException;

import org.testng.annotations.Test;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.github.javafaker.Faker;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import pojo.Comments;

public class CommentsApiTest {
   @Test
   public void createCommentFromFixtureWithDynamicData() throws IOException {
      File fixture = new File(
         "src/test/resources/payload/commentsPostPayload.json");
      ObjectMapper mapper = new ObjectMapper();
      Comments comments = mapper.readValue(fixture, Comments.class);

      Faker faker = new Faker();
      comments.setName(faker.name().fullName());
      comments.setEmail(faker.internet().emailAddress());
      comments.setBody("QA automation: " + faker.lorem().sentence());

      String requestJson = mapper.writeValueAsString(comments);

      Response response = given()
         .contentType(ContentType.JSON)
         .accept(ContentType.JSON)
```

```
        .body(requestJson)
    .when()
        .post("https://jsonplaceholder.typicode.com/comments");

    response.then()
        .statusCode(201)
        .contentType(ContentType.JSON)
        .body("postId", equalTo(comments.getPostId()))
        .body("name", equalTo(comments.getName()))
        .body("email", equalTo(comments.getEmail()))
        .body("id", notNullValue());
  }
}
```

4.4 Code walkthrough

Code / method	Meaning
readValue(fixture, Comments.class)	Loads the JSON test data into a Java object.
new Faker()	Creates a data generator for example names and emails.
comments.setName(...)	Replaces only the name field; other fixture values remain unless changed.
writeValueAsString(comments)	Converts the current object back into JSON.
contentType(ContentType.JSON)	Tells the API that the request body is JSON.
accept(ContentType.JSON)	Requests a JSON response where the server supports content negotiation.
body(requestJson)	Attaches the serialized JSON string to the request.
post(url)	Sends an HTTP POST request.
statusCode(201)	Checks the API's documented creation response code.
body("id", notNullValue())	Checks that the response includes an id field.

Demo API limitation: JSONPlaceholder is commonly used as a mock/demo API. A successful POST response can look like a create, but the record may not be stored for a later GET. Do not write a follow-up persistence assertion unless the target API contract supports it.

5. Faker: generating test data safely

5.1 Useful methods from the class notes

```
Faker faker = new Faker();

String firstName = faker.name().firstName();
String lastName  = faker.name().lastName();
String fullName  = faker.name().fullName();
String email     = faker.internet().emailAddress();
String city      = faker.address().city();
String state     = faker.address().state();
String country   = faker.address().country();
String zipCode   = faker.address().zipCode();
```

Each result is generated at runtime. The exact value varies between runs. The source file shows four address examples but reuses the variable name city several times; use separate variable names (city, state, country, zipCode) so the Java code compiles and the intent is clear.

5.2 More controlled data

```
// Example: unique email for a create-user test
String email = "qa+" + UUID.randomUUID().toString().replace("-", "")
        + "@example.test";

// Use faker for descriptive non-unique values
String firstName = faker.name().firstName();
String city = faker.address().city();
```

Some APIs enforce uniqueness on email, username or phone. Pure random data can collide eventually. A UUID suffix is a practical way to make an identifier unique within a test run. Check the API's email-validation rules before using reserved test domains such as example.test.

5.3 Static fields versus dynamic fields

Field	Possible approach	Reason
firstName / lastName	Faker	Good for avoiding repeated human-name values.
email / username	Unique suffix or faker plus uniqueness	APIs often enforce unique values.
country / city	Faker or curated list	Faker values may not satisfy application-specific allowed lists.
mobileNumber	String fixture or controlled generator	Phone numbers are identifiers, not quantities; leading zeros and + may matter.
password	Safe test secret from configuration	Do not use Faker output as a production credential or print it in logs.
roles / projectName	Curated fixture values	Often restricted to a fixed set of valid values.

5.4 Reproducibility

When a random test fails, save the generated payload (after masking sensitive values) so the failure can be reproduced. For deterministic runs, use a known fixture or seed Faker if supported by the version in your project. Do not assume generated values are repeatable across library versions.

Security rule: Never log a request containing passwords, session tokens, authorization headers, or personal information without an approved masking policy. The source examples use request.log().all(); keep that for non-sensitive demos only.

6. Example 2 — Dynamic Employee request

6.1 Starting fixture

```
{
 "firstName": "suraj",
 "lastName": "kadam",
 "email": "suraj@gmail.com",
 "mobileNumber": "9090901122",
 "city": "pune",
 "country": "India",
 "projectName": "BSS",
 "username": "suraj@gmail.com",
 "password": "Suraj@123"
}
```

The upload initially shows mobileNumber as a number in one JSON example and as long in one POJO. Prefer String for phone/mobile values: a phone number is not used for arithmetic and may have leading zeros, plus signs or formatting.

6.2 Employee POJO

```
package pojo;

import lombok.Data;

@Data
public class Employees {
    private String firstName;
    private String lastName;
    private String email;
    private String mobileNumber;
    private String city;
    private String country;
    private String projectName;
    private String username;
    private String password;
}
```

6.3 Test flow based on the uploaded code

```
File fixture = new File(
    "src/test/resources/payload/employeePostRequestBody.json");
ObjectMapper mapper = new ObjectMapper();
Employees employee = mapper.readValue(fixture, Employees.class);

Faker faker = new Faker();
employee.setFirstName(faker.name().firstName());
employee.setLastName(faker.name().lastName());
employee.setEmail(faker.internet().emailAddress());
employee.setCity(faker.address().city());
employee.setCountry(faker.address().country());

String json = mapper.writeValueAsString(employee);
Response response = given()
    .contentType(ContentType.JSON)
    .body(json)
.when()
    .post("http://localhost:9192/api/employees");

response.then()
    .statusCode(201)
    .body("id", notNullValue())
    .body("email", equalTo(employee.getEmail()));
```

Expected status should match the local API contract. If your service returns 200 instead of 201, use the documented value; do not change the assertion simply to make a failing test pass.

6.4 Why keep reading from a fixture?

- You keep required properties that the API expects.
- You only replace fields needed for uniqueness or scenario variation.
- It is easier to review changes than constructing every field from scratch.
- You can maintain separate fixtures for valid, missing-field and boundary-value cases.

6.5 Do not mutate a shared fixture file

Read the JSON file and update the in-memory POJO. Do not rewrite the fixture file during each test. If the test changes the stored fixture, later tests can become order-dependent and the repository will be left with random data.

Data consistency check: The fixture uses email as both email and username. The example changes email dynamically but leaves username unchanged. If your API requires username and email to match or requires both to be unique, generate one unique value and assign it to both fields; otherwise follow the API contract.

7. Nested POJO example — User payload

7.1 Why nested classes are needed

A JSON object may contain another object. In the sample user payload, address contains geolocation and name is another nested object. The Java model should preserve that structure rather than flatten every property into one class.

```
{
 "address": {
  "geolocation": { "lat": "-37.3159", "long": "81.1496" },
  "city": "kilcoole",
  "street": "new road",
  "number": 7682,
  "zipcode": "12926-3874"
 },
 "email": "john@example.test",
 "username": "johnd",
 "password": "example-only",
 "name": { "firstname": "john", "lastname": "doe" },
 "phone": "1-570-236-7033",
 "__v": 0
}
```

7.2 POJO classes

```
// Name.java
@Data
public class Name {
    private String firstname;
    private String lastname;
}

// Geolocation.java
import com.fasterxml.jackson.annotation.JsonProperty;

@Data
public class Geolocation {
    private String lat;

    @JsonProperty("long")
    private String geoLong;
}

// Address.java
@Data
public class Address {
    private Geolocation geolocation;
    private String city;
    private String street;
    private Integer number;
    private String zipcode;
```

```
}

// Users.java
@Data
public class Users {
    private Address address;
    private String email;
    private String username;
    private String password;
    private Name name;
    private String phone;
}
```

7.3 Special field names and mapping

Java property names should follow Java naming conventions. The JSON key long is awkward as a Java identifier, and __v is an API-specific field name. @JsonProperty tells Jackson which JSON property should map to a Java field. If a response includes __v, map it in a response DTO. Omit server-managed fields from a create-request DTO unless the API contract explicitly requires them.

7.4 Build the object graph

```
Name name = new Name();
name.setFirstname(faker.name().firstName());
name.setLastname(faker.name().lastName());

Geolocation geo = new Geolocation();
geo.setLat("-37.3159");
geo.setGeoLong("81.1496");

Address address = new Address();
address.setGeolocation(geo);
address.setCity(faker.address().city());
address.setStreet(faker.address().streetAddress());
address.setZipcode(faker.address().zipCode());
address.setNumber(123);

Users user = new Users();
user.setName(name);
user.setAddress(address);
user.setEmail(faker.internet().emailAddress());
user.setUsername("qa_" + UUID.randomUUID().toString().replace("-", ""));
user.setPassword("Use-a-test-secret-from-config");
user.setPhone("+1-555-0100");

String json = mapper.writerWithDefaultPrettyPrinter()
                .writeValueAsString(user);
```

Objects are assembled from the inside out: create Name and Geolocation first, attach them to Address and Users, then serialize the root Users object. Jackson walks the nested references and emits nested JSON.

8. Complete nested User POST example

```
package tests;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.notNullValue;
import static org.hamcrest.Matchers.equalTo;

import java.io.IOException;
import java.util.UUID;

import org.testng.annotations.Test;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.github.javafaker.Faker;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import pojo.Address;
import pojo.Geolocation;
import pojo.Name;
```

```
import pojo.Users;

public class UserApiTest {
   @Test
   public void createUserWithNestedPayload() throws IOException {
      Faker faker = new Faker();
      ObjectMapper mapper = new ObjectMapper();

      Name name = new Name();
      name.setFirstname(faker.name().firstName());
      name.setLastname(faker.name().lastName());

      Geolocation geo = new Geolocation();
      geo.setLat("-37.3159");
      geo.setGeoLong("81.1496");

      Address address = new Address();
      address.setGeolocation(geo);
      address.setCity(faker.address().city());
      address.setStreet(faker.address().streetAddress());
      address.setZipcode(faker.address().zipCode());
      address.setNumber(123);

      Users user = new Users();
      user.setName(name);
      user.setAddress(address);
      user.setEmail(faker.internet().emailAddress());
      user.setUsername("qa_" + UUID.randomUUID().toString().replace("-", ""));
      user.setPassword("test-only-secret-from-config");
      user.setPhone("+1-555-0100");

      String json = mapper.writeValueAsString(user);
      Response response = given()
         .contentType(ContentType.JSON)
         .accept(ContentType.JSON)
         .body(json)
      .when()
         .post("https://fakestoreapi.com/users");

      response.then()
         .statusCode(200) // Verify the target API contract before using this value.
         .body("id", notNullValue());
   }
}
```

Contract-dependent assertion: Demo APIs can differ in their response codes and fields. The status code above is shown as an example placeholder for a service that responds with 200. Confirm your target API's documented response before running; do not assume every POST returns the same code.

8.1 What to assert in the nested response

```
response.then()
   .body("name.firstname", equalTo(user.getName().getFirstname()))
   .body("address.city", equalTo(user.getAddress().getCity()))
   .body("address.geolocation.lat", equalTo("-37.3159"));
```

Dot notation targets a nested JSON field. If the API returns a transformed representation, compare against the contract rather than assuming it echoes every request field exactly.

9. POJO mapping and model design best practices

9.1 Choose types that match the contract

JSON concept	Common Java type	Guidance
Text, email, phone, postal code	String	Phone numbers and postal codes are identifiers; preserve + and leading zeroes.
Whole-number count	Integer / Long	Use wrappers if null/missing is

JSON concept	Common Java type	Guidance
		meaningful. Choose range based on contract.
Money/price	BigDecimal	Avoid float/double for financial precision.
True/false	Boolean / boolean	Use Boolean when null is distinct from false.
Nested JSON object	Another POJO	Keep hierarchy aligned with JSON.
JSON array of text	List<String>	Make collection type reflect the item type.
JSON array of objects	List<Avatar>	Create a class for each array item shape.

9.2 Request DTOs versus response DTOs

For small learning examples, one POJO may be reused for request and response. In an industry framework, prefer separate request and response DTOs when the shapes differ. A response may include server-generated id, timestamps, internal version fields or status metadata that the create request should not send.

9.3 Avoid overusing @Data

@Data generates getters, setters, toString, equals and hashCode. It is convenient for test DTOs. However, toString may expose password or personally identifiable values if printed. For sensitive DTOs, use @Getter/@Setter selectively or override the string representation to mask sensitive fields.

9.4 Lombok setup reminders

- Add the Lombok dependency with scope appropriate for your project.
- Enable annotation processing in the IDE/build as required by that IDE.
- Restart/reimport Maven if generated getters/setters appear missing.
- If an IDE reports an error while Maven builds successfully, check IDE annotation-processing setup before changing the source code.

9.5 Jackson mapping rules

- JSON property names should match Java property names unless you use @JsonProperty or a naming strategy.
- Make sure nested POJOs and collection item types match the JSON structure.
- If a JSON field can be absent or null, use a nullable wrapper type where it is important to distinguish null from a default primitive value.
- Configure unknown-property behavior deliberately; do not ignore contract drift silently without a reason.

10. Negative testing and real-world scenarios

Scenario	Setup	What to verify
Valid create	Use fixture plus valid dynamic email	Documented success status, generated id, returned values.
Duplicate email	Reuse an email already registered	Business validation error; no duplicate created.
Missing required field	Set email or firstName to null/remove it	Documented 4xx response and useful error details.
Invalid email	Use a deliberately malformed address	Validation failure; no server error 500.
Bad nested object	Send address without required city or geolocation	Validation response points to invalid structure.
Unsupported type	Send string where numeric field is required	400/422 or documented validation error.
API unavailable	Stop local service or use wrong port	Clear connection failure diagnosis; not misreported as assertion failure.
Unexpected response shape	Remove expected response key in mock/stub	Assertion or schema validation catches contract drift.
Sensitive data leaked	Request contains password and request logging enabled	Ensure logs mask/remove password and Authorization header.
Repeatability	Run test repeatedly and in parallel	No collision from username/email; tests clean up created records.

10.1 Assert the whole outcome, not just status

```
response.then()
   .statusCode(201)
   .contentType(ContentType.JSON)
   .body("id", notNullValue())
   .body("email", equalTo(employee.getEmail()))
   .body("firstName", equalTo(employee.getFirstName()));
```

A status code alone is not enough. A create endpoint might return 201 while a required field is missing from the response or the wrong email was stored. Add assertions for the business values that matter.

10.2 TestNG pattern

```
@Test
public void createEmployeeShouldReturnIdAndEchoEmail() throws IOException {
   // Arrange: load fixture, map to POJO, override email.
   // Act: send POST request.
   // Assert: status code and business fields.
}
```

Arrange-Act-Assert keeps tests readable: arrange test data, perform one API action, then assert the outcome. In a full framework, move repeated payload-building and API-client code into helpers, but keep each test's purpose obvious.

11. Common mistakes and how to fix them

Problem	Why it happens	Fix
Wrong file path	Absolute path belongs to one developer's machine.	Use src/test/resources or classpath resource loading.
Faker variable redeclared	Class notes use String city repeatedly for city/state/country/zip.	Use distinct local variable names.
POJO field does not populate	JSON and Java property names do not match.	Rename field or use @JsonProperty.
Missing nested object	Parent POJO is created but child object is not attached.	Set address.setGeolocation(geo), then users.setAddress(address).
Primitive becomes 0	Missing JSON numeric property maps to Java int default.	Use Integer if null/missing matters and assert required properties.
Phone number loses leading zero	Phone saved as numeric type.	Use String for phone/mobile values.
Random test failure hard to reproduce	Faker generates a different payload on each run.	Log a safe correlation id and store a sanitized failure payload; use fixed fixture or deterministic seed when appropriate.
Duplicate user fails intermittently	Email/username generated values collide.	Use a unique suffix and cleanup created test data.
Password appears in console	request.log().all() prints request body.	Avoid body/header logging for secrets; mask sensitive fields.
Wrong expected status code	Assumption copied from a different API.	Check Swagger/OpenAPI or API contract and assert the documented status.

11.1 Exceptions to recognise

- FileNotFoundException / IOException: path is wrong or the fixture cannot be read; check location and permissions.
- JsonMappingException: JSON structure or a value cannot be mapped to the target POJO; check data types and nested class structure.
- JsonParseException: JSON syntax is malformed; check braces, quotes, commas and literals.
- AssertionError: the API responded but one of your expected values did not match; inspect the response and contract.
- Connection refused / timeout: service is unavailable or URL/port is wrong; fix environment before interpreting it as a functional assertion failure.

12. Best-practice checklist

- Keep JSON fixtures under src/test/resources and name them by scenario, for example createEmployee-valid.json.
- Keep fixture files immutable during test execution. Update only the POJO instance in memory.
- Use POJOs for structured payloads; use raw JSON only when deliberately testing malformed or special cases.
- Generate dynamic values only for fields that need to be unique or varied.
- Use String for phone, postal code and identifier fields; use BigDecimal for money when precision matters.
- Use ObjectMapper consistently; reuse a configured mapper in a test utility rather than creating dozens of slightly different configurations.
- Use ContentType.JSON rather than spelling the media type repeatedly.
- Assert response status, content type, generated id and important business fields.
- Check the API contract before assuming a POST returns 200 or 201.
- Mask passwords, access tokens, authorization headers and sensitive data from logs.
- Use environment variables or CI secret storage for credentials; never commit real secrets into fixture files.
- Use independent tests when possible. Avoid relying on test order unless the workflow intentionally tests a multi-step process.
- Generate unique data for parallel runs and clean up created entities when the environment supports it.
- On a failed dynamic test, preserve enough sanitized evidence to reproduce the issue.
- Keep request DTOs focused on writable fields. Do not send server-managed id/version/timestamps unless the API requires them.
- Use positive, negative, boundary and contract tests; random happy-path testing is not sufficient.
- For public demo APIs, treat remote availability and persistence as uncertain; use local mocks/stubs for deterministic CI where possible.

13. Hands-on exercises

Try each exercise before reading the worked solutions. Use the payload file and POJO pattern from the chapters above.

Exercise 1 — Static Comments POST

Send commentsPostPayload.json directly as the request body without Jackson or Faker. Assert the documented success status and JSON response content type.

Exercise 2 — Dynamic comment author

Deserialize the Comments fixture, replace only name and email with Faker values, serialize it, and assert the response contains the generated email.

Exercise 3 — Unique employee

Create an employee request with a unique email. Verify generated id, firstName and email. Make sure the mobile number remains a String.

Exercise 4 — Negative email

Set email to "not-an-email" and assert the API returns the documented validation response. Verify the API does not return an unexpected server error.

Exercise 5 — Preserve invariant data

Modify only firstName, lastName, email, city and country. Assert that projectName and other fixture fields remain unchanged in the serialized JSON.

Exercise 6 — Nested user

Build Users, Name, Address and Geolocation POJOs. Serialize the root object and verify the produced JSON contains address.geolocation.lat and name.firstname.

Exercise 7 — @JsonProperty

Map Java field geoLong to JSON key long. Serialize the object and confirm the JSON key is exactly "long", not "geoLong".

Exercise 8 — Repeatability

Run the test 10 times. Use unique emails and record only a safe test correlation value; confirm there are no duplicate-email failures.

Exercise 9 — Fixture path

Move a fixture into src/test/resources and replace any absolute path. Verify the test runs on another developer's machine or in CI with the same repository structure.

Exercise 10 — Secret-safe logging

Find every request that contains password or credentials. Remove request body logging or add a masking strategy. Confirm the console does not reveal the secret.

14. Worked solutions (selected)

Solution 1 — Static request

```
File fixture = new File(
   "src/test/resources/payload/commentsPostPayload.json");
Response response = given()
   .contentType(ContentType.JSON)
   .body(fixture)
.when()
   .post("https://jsonplaceholder.typicode.com/comments");
response.then()
   .statusCode(201)
   .contentType(ContentType.JSON);
```

Solution 2 — Change only two fields

```
Comments comments = mapper.readValue(fixture, Comments.class);
String originalBody = comments.getBody();
Faker faker = new Faker();
comments.setName(faker.name().fullName());
comments.setEmail(faker.internet().emailAddress());
assert comments.getBody().equals(originalBody);
String json = mapper.writeValueAsString(comments);
```

Solution 3 — Check the nested JSON path

```
String json = mapper.writeValueAsString(user);
JsonNode root = mapper.readTree(json);
assert root.path("name").path("firstname").isTextual();
assert root.path("address").path("geolocation").path("lat").isTextual();
```

These assertions check the structure before sending the request. In TestNG, use Assert.assertTrue(...) rather than Java assert unless assertions are explicitly enabled; Java's assert statement is commonly disabled by default.

Solution 4 — Correctly name Faker variables

```
String city = faker.address().city();
String state = faker.address().state();
String country = faker.address().country();
String zipCode = faker.address().zipCode();
```

Solution 5 — Why a mobile number is String

A mobile number is not a value to add, divide or average. Numeric storage can remove leading zeroes and cannot naturally represent a plus sign or extensions. Keep it as String in the POJO and JSON request if that matches the service contract.

15. Mini capstone — Dynamic employee creation

Goal: create a maintainable test that uses the supplied employee fixture as a template, changes unique values, submits a POST, validates the response, and leaves the original JSON file unchanged.

Acceptance criteria

- Fixture exists and can be deserialized to Employees.
- firstName, lastName, email, city and country are changed in memory.
- email or username is unique per execution where the API requires uniqueness.
- Request is sent as application/json.
- Response status matches the API contract.
- Generated id and important returned values are asserted.
- Logs do not contain passwords or sensitive request data.
- The test can execute repeatedly without depending on prior test state.

Implementation skeleton

```
@Test
public void createEmployeeFromDynamicFixture() throws IOException {
   File fixture = new File(
      "src/test/resources/payload/employeePostRequestBody.json");
   ObjectMapper mapper = new ObjectMapper();
   Employees employee = mapper.readValue(fixture, Employees.class);

   Faker faker = new Faker();
   employee.setFirstName(faker.name().firstName());
   employee.setLastName(faker.name().lastName());
   employee.setEmail("qa+" + UUID.randomUUID().toString().replace("-", "")
      + "@example.test");
   employee.setCity(faker.address().city());
   employee.setCountry(faker.address().country());

   String payload = mapper.writeValueAsString(employee);
   Response response = given()
      .contentType(ContentType.JSON)
      .body(payload)
   .when()
      .post("http://localhost:9192/api/employees");

   response.then()
      .statusCode(201) // confirm against your local API contract
      .body("id", notNullValue())
      .body("email", equalTo(employee.getEmail()));
}
```

Capstone note: If your local service returns 200 or 202 rather than 201, update the expectation only after checking its Swagger/OpenAPI definition or agreed contract. If it does not echo email or expose id at the root, adapt the assertions to its actual response structure.

16. Interview revision questions

Q: What is serialization?
Converting a Java object into a JSON representation, for example ObjectMapper.writeValueAsString(object).

Q: What is deserialization?
Converting JSON into a Java object, for example ObjectMapper.readValue(file, Comments.class).

Q: Why load JSON into a POJO before using Faker?
It preserves an existing valid payload structure while allowing selected values to change.

Q: Can Rest Assured accept a File, POJO, or JSON String in body()?
Yes, depending on configured object mappers and supported body types. A String should already be valid JSON when Content-Type is application/json.

Q: Why use Faker?
To generate varied test data that reduces collisions and broadens input coverage.

Q: Does Faker guarantee uniqueness?
No. Use explicit unique suffixes or a test-data strategy for unique fields.

Q: Why should phone numbers be strings?
They are identifiers, can have leading zeros and symbols, and should not be used in numeric calculations.

Q: Why use nested POJOs?
They mirror nested JSON and make payload creation and assertions easier to understand.

Q: When is @JsonProperty useful?
When JSON property names differ from Java field names, such as JSON long and Java geoLong.

Q: Is logging every request always a good practice?
No. It may leak passwords, tokens or personal data. Use masked/limited logging.

Q: Should a test change the stored fixture file?
Generally no. Deserialize it and modify the POJO in memory.

Q: Does a 201 status alone prove a correct create?
No. Also check the generated ID and the business fields important to the contract.

17. Final quick reference

Task	Pattern
Read JSON file into POJO	mapper.readValue(file, Comments.class)
Generate sample name	faker.name().fullName()
Generate sample email	faker.internet().emailAddress()
Change a field	pojo.setEmail(generatedEmail)
Convert POJO to JSON	mapper.writeValueAsString(pojo)
Pretty-print JSON	mapper.writerWithDefaultPrettyPrinter().writeValueAsString(pojo)
Send JSON body	given().contentType(ContentType.JSON).body(json).post(url)
Check created resource	status code + id + important response fields
Nested JSON field	body("address.city", equalTo(expectedCity))
JSON file location	src/test/resources/payload/yourPayload.json

Final workflow to remember

```
Static JSON fixture
  -> ObjectMapper.readValue(...)
  -> POJO object
  -> Faker updates selected fields
  -> ObjectMapper.writeValueAsString(...)
  -> Rest Assured POST request
  -> Assert status, response shape and business fields
```

The key idea from Class 8 is that a fixture gives you a known starting point, POJOs give your data structure, Faker supplies varied values, Jackson converts between JSON and Java, and Rest Assured sends the request. Assertions determine whether the API behaved according to its contract.

Source alignment: These notes follow the topics and examples in your uploaded Class 8 file: static versus dynamic payloads, Comments, Employee, nested User objects, JavaFaker, Jackson ObjectMapper, and Rest Assured POST requests. Corrections and best-practice recommendations are identified as teaching guidance where they go beyond the source.